

FREE SAMPLER · 8 COMPLETE CHAPTERS

PyTorch From Ground Up

Volume 1: Foundations

Most tutorials are building a model by chapter two, and leave you quietly confused. Why unsqueeze here? When do you need keepdim? Why turn autograd off to optimize?

This book slows down and builds everything from the tensor up, so nothing stays hazy. It is the first volume in a series working toward large language models — foundations first, because you cannot reason about an LLM if the tensor underneath it is vague.

These are eight complete, unabridged chapters from across the book — not a front-of-book teaser. Nothing has been cut.

Wesam

Electronics & control engineer — 25 years in industrial control systems

The eight chapters

Chosen from across all four parts of the book, not the easy opening run.

2 What a Tensor Really Is
The foundation everything else rests on.

3 Your First Complete Model
A model that learns an equation it was never shown.

5 Indexing and Slicing
Constantly needed, rarely taught properly.

13 Gradients Intuitively
The idea most people quietly fake.

19 nn.Module
The gate between tensor work and real PyTorch.

23 Dataset
Where beginners stall once the tutorials end.

27 Anatomy of a Training Loop
Every line of the loop, explained.

34 Debugging PyTorch
Reading the errors instead of guessing.

PART I · TENSOR FLUENCY — CHAPTER 2

What a Tensor Really Is

A tensor is just a box of numbers with a shape. Everything else follows from that.

The word *tensor* can sound intimidating — it shows up in physics, differential geometry, and a lot of heavy mathematics. Forget all of that. In PyTorch, a tensor is the simplest thing in the world: a container of numbers arranged in a regular grid, with a shape that tells you how those numbers are organized. A single number is a tensor. A row of numbers is a tensor. A spreadsheet of numbers is a tensor. A stack of spreadsheets is a tensor. The only thing that changes is how many dimensions the grid has.

This chapter walks you through that idea from the ground up: one dimension at a time, then the three facts PyTorch keeps about every tensor — its *shape*, its *dtype*, and its *device* — and, finally, the quiet truth underneath: how those numbers actually sit in memory, and what a *stride* is. You do not need to memorize the memory model today, but meeting it now means it will feel familiar the first time a reshape fails and you need to understand why.

The dimension ladder

Every tensor has a number of dimensions, which PyTorch calls its `ndim`. The ladder below shows the first four rungs. There is no upper limit — you can have five, six, or more dimensions — but almost everything you meet in practice lives on these four.

7

Scalar · 0-D

shape: ()

A single number. No axes at all.

1 2 3 4

Vector · 1-D

shape: (4,)

A row of numbers. One axis.

1 2 3 4

5 6 7 8

Matrix · 2-D

shape: (2, 4)

Rows and columns. Two axes.

a b c g h i

d e f j k l

3-D tensor

shape: (2, 2, 3)

A stack of matrices. Three axes.

Each rung adds one axis. A 3-D tensor is just two matrices stacked; a 4-D tensor is a batch of those stacks.

```
scalar = torch.tensor(7)
vector = torch.tensor([1, 2, 3, 4])
matrix = torch.tensor([[1,2,3,4],
                        [5,6,7,8]])
cube   = torch.randn(2, 2, 3)

print(scalar.ndim, vector.ndim,
      matrix.ndim, cube.ndim)
```

OUTPUT

0 1 2 3

TRY IT

Create a 4-D tensor with `torch.zeros(2, 3, 4, 5)`. Print its `.ndim` and `.shape`. Can you say in plain English what each of the four numbers in the shape means — how many of *what*?

The three facts: shape, dtype, device

Every tensor carries exactly three pieces of metadata, and checking them is how you stay out of trouble. You already met *device* in Chapter 1. Now we add the other two.

```
t = torch.tensor([[1.0, 2.0],
                  [3.0, 4.0]])

print(t.shape)    # torch.Size([2, 2])
print(t.dtype)    # torch.float32
print(t.device)   # cpu
```

Shape tells you how the numbers are arranged: two rows, two columns. **Dtype** (data type) tells you what kind of number each cell holds — here, a 32-bit floating-point number. **Device** tells you where the tensor lives — on the CPU or on a GPU. Together, these three facts answer the question "what am I working with?" and they are the first thing to print whenever something goes wrong.

Dtype: picking the right number type

Not every number needs the same amount of precision. PyTorch lets you choose, and the default — `float32` — is the right one for almost everything you will do in this book. Here are the types you will actually encounter:

DTYPE	WHAT IT STORES	WHEN YOU'LL USE IT
<code>torch.float32</code>	32-bit decimal	Default. Model weights, inputs, losses.
<code>torch.float64</code>	64-bit decimal	Rare. Extra precision for numerical checks.
<code>torch.float16</code>	16-bit decimal	Mixed-precision training on GPUs.
<code>torch.int64</code>	64-bit integer	Class labels, indices.
<code>torch.bool</code>	True / False	Masks, comparisons.

You can always check with `.dtype` and convert with `.to(torch.float32)` or `.float()`.

```
labels = torch.tensor([0, 1, 2])
print(labels.dtype)          # torch.int64

floats = labels.float()      # convert to float32
print(floats.dtype)         # torch.float32
```

WATCH OUT

When you create a tensor from integers like `[1, 2, 3]`, PyTorch defaults to `int64`. When you create from decimals like `[1.0, 2.0]`, it defaults to `float32`. Mixing the two in an operation often produces a dtype-mismatch error. The fix: cast one of them explicitly with `.float()` or `.long()`.

Shape: learning to read it

The shape is a tuple of integers, one per dimension, and it tells you the size along each axis. Reading it is a skill you will use a hundred times a day, so it is worth getting comfortable now.

```
img_batch = torch.randn(32, 3, 28, 28)
print(img_batch.shape)
```

OUTPUT

```
torch.Size([32, 3, 28, 28])
```

Read it left to right: 32 images, each with 3 color channels, each channel 28 pixels tall and 28 pixels wide. This convention — *batch, channels, height, width*, or **NCHW** — is the standard layout for image data in PyTorch, and you will see it constantly.

```
print(img_batch.shape[0])    # 32 – batch size
print(img_batch.shape[-1])   # 28 – last dim (width)
print(img_batch.size(1))     # 3 – channels
```

You can index into `.shape` like any tuple, and `.size(dim)` does the same thing. Both are useful; use whichever reads more clearly in context.

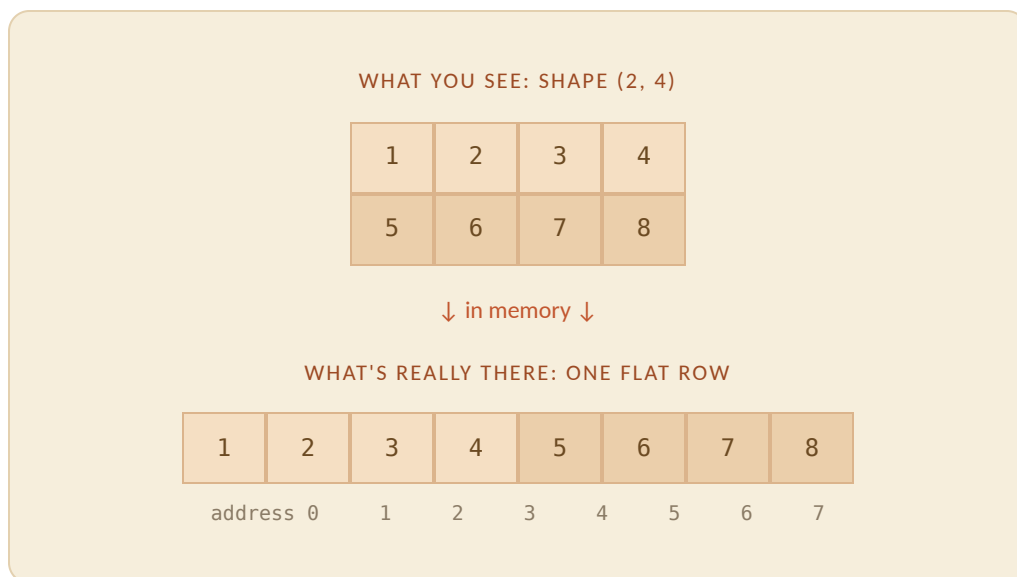
TRY IT

Create `torch.randn(8, 512)`. Without running it, say what `.shape`, `.ndim`, and `.shape[0]` will print. Then run it and check.

Under the hood: memory is flat

Here is the quiet truth that makes reshape, view, transpose, and contiguity all make sense: no matter how many dimensions a tensor *looks* like it has, the actual numbers sit in a single, flat row in memory. A 2×4

matrix is eight numbers in a line. The shape is just a lens PyTorch holds over that line to let you see it as rows and columns.



Row-major order: row 0 first, then row 1, laid out end to end.

PyTorch (like C and NumPy) stores tensors in *row-major* order: the last axis varies fastest. So the first row fills addresses 0–3, the second row fills 4–7. This is just a convention, but it matters because it determines which operations can reinterpret the memory without copying it (like `view`) and which cannot.

Strides: how PyTorch navigates the flat row

If memory is flat, how does PyTorch find the element at row 1, column 2? With a small, elegant idea called a *stride*. The stride tells you how many positions to jump in memory to move one step along each axis.

STRIDE ALONG ROWS = 4

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

jump 4 positions to reach the next row

STRIDE ALONG COLUMNS = 1

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

jump 1 position to reach the next column

For a (2, 4) tensor stored row-major, the strides are (4, 1).

```
m = torch.tensor([[1,2,3,4],
                  [5,6,7,8]])

print(m.shape)      # torch.Size([2, 4])
print(m.stride())   # (4, 1)
```

The tuple (4, 1) says: to move one step down (next row), jump 4 elements; to move one step right (next column), jump 1. That is all PyTorch needs to turn a flat memory block into any shape you ask for. You will see strides again when we talk about **view** vs **reshape**, **transpose**, and contiguity — they are the single idea underneath all of those.

WATCH OUT

You do not need to work with strides directly in everyday code. They exist so that operations like **transpose** and **view** can rearrange a tensor without copying its data — which is fast and memory-efficient, but occasionally means a "non-contiguous" tensor cannot be **view**-ed, only **reshape**-ed. We will cover that in detail in Chapter 6.

Numel and storage size

Two quick helpers round out the picture. `.numel()` returns the total number of elements — the product of all dimensions — and `.element_size()` tells you how many bytes each one takes.

```
t = torch.randn(3, 4)

print(t.numel())           # 12  (3 × 4)
print(t.element_size())    # 4 bytes (float32)
print(t.numel() * t.element_size(),
      "bytes")             # 48 bytes total
```

These are useful later when you want to estimate how much memory a model or a batch of data will consume — and they give you a concrete sense of scale that shape alone does not.

Putting it all together

Here is the complete mental model so far. A tensor is a grid of numbers. PyTorch remembers three things about it: its *shape* (how the grid is organized), its *dtype* (what kind of number), and its *device* (where it lives). Under the hood, the grid is really a flat row of numbers, and the *stride* is the recipe for navigating it. That is the whole foundation — everything from here forward is just learning to work with it fluently.

THE RULE

- 1 A tensor is a grid of numbers with a *shape* — one integer per axis.
- 2 Every tensor carries three facts: **shape**, **dtype**, **device**. Print them first when debugging.
- 3 **ndim** counts the axes. **numel()** counts the total elements.
- 4 Memory is flat. The **stride** tells PyTorch how to navigate it — which is why some reshapes are free and others require a copy.
- 5 The default dtype is **float32** for decimals, **int64** for integers. Cast explicitly when mixing them.

END OF CHAPTER 2

Exercises

1. Create tensors of dimensions 0 through 4 and print each one's **.ndim**, **.shape**, and **.numel()**. Verify that **numel** equals the product of the shape.
2. Create **torch.tensor([1, 2, 3])** and **torch.tensor([1.0, 2.0, 3.0])**. Print each tensor's **.dtype**. Why are they different? Convert the integer tensor to **float32** and confirm the dtype changed.
3. Create a (3, 5) tensor of random numbers. Print its **.stride()**. Explain in one sentence what each number in the stride tuple means.
4. Calculate by hand how many bytes a **torch.float32** tensor of shape (64, 3, 224, 224) would consume. Then check with **.numel() * .element_size()**. Does your answer match?
5. Create a (2, 3) tensor. Print its **.storage()** — what do you notice about the format? How does it compare to what you see when you print the tensor normally?

INTERVIEW-STYLE PROBLEM

An interviewer asks: "You have a tensor of shape **(32, 3, 28, 28)**. What does each dimension represent, and how many total numbers does this tensor contain?" Answer in two to three sentences. Then they follow up: "What is its stride, and why does the stride matter?" Give a one-sentence answer.

PART I · TENSOR FLUENCY — CHAPTER 3

Your First Complete Model

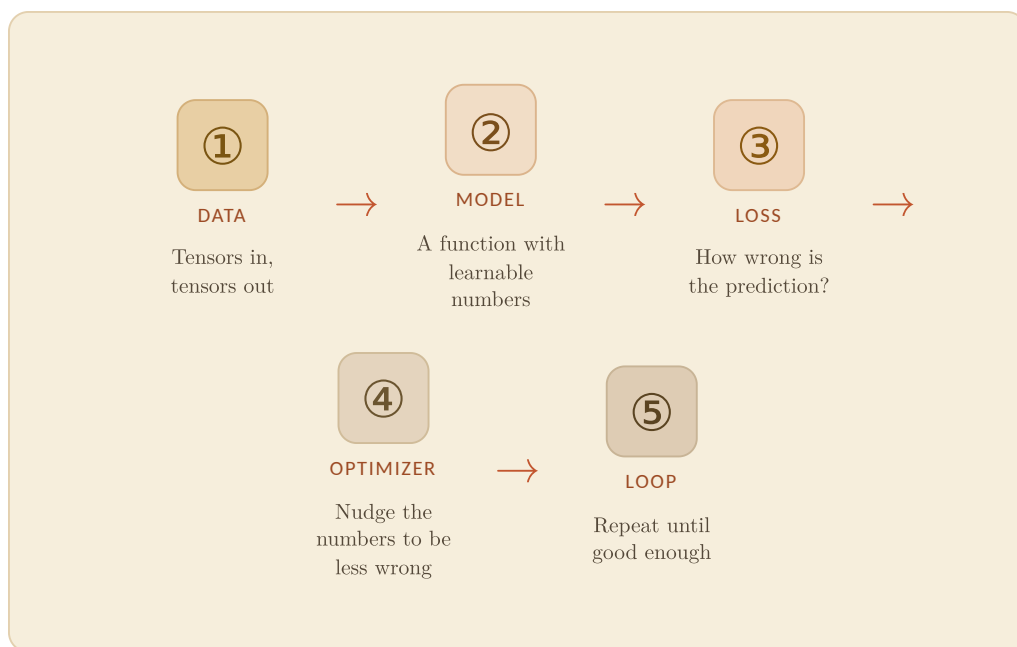
See the whole pipeline once — data, model, loss, optimizer, training loop — before we explain every piece.

We are about to do something slightly reckless: build and train a complete working model before most of the vocabulary has been properly introduced. The point is not to understand every line today. The point is to *see the whole thing run*, from raw numbers to a model that actually learns, so that every piece we study in the chapters ahead has a place to land in your head. Think of this chapter as a movie trailer for the rest of the book.

The problem we will solve is deliberately tiny: given some numbers, learn to predict a simple pattern. The model is a single line. The training loop fits on a screen. And by the end of it, the error will fall from high to nearly zero — which is what learning looks like, stripped down to its skeleton.

The five-part pipeline

Every model you will ever build in PyTorch, from a one-line toy to a billion-parameter language model, follows the same five steps. We will name them now and fill them in with real code in a moment.



Data → Model → Loss → Optimizer → Repeat. Every PyTorch project is a variation of this.

That is the entire architecture of machine learning: make a prediction, measure the error, adjust, and repeat. The loop is the heartbeat; everything else is plumbing. Let us build each piece.

Step 1 — the data

We need something to learn from. Our toy problem: given x , predict y , where the true relationship is $y = 3x + 1$. The model will never see that formula — it will discover the pattern from the numbers alone.

```
import torch

torch.manual_seed(0)

# 20 training points, x from 0 to 2
x = torch.linspace(0, 2, 20).unsqueeze(1)  # (20, 1)
y = 3 * x + 1 + 0.3 * torch.randn_like(x) # noisy

print(x.shape, y.shape)
```

OUTPUT

```
torch.Size([20, 1]) torch.Size([20, 1])
```

Twenty pairs of (x, y) , each a tiny tensor of shape $(20, 1)$. The noise means the points scatter around the true line, so the model cannot simply memorize — it has to find the underlying trend. Don't worry about `unsqueeze` or `randn_like` yet; we will cover both properly later. What matters is that we now have tensors of input and tensors of output.

Step 2 — the model

Our model is a single linear layer: it takes one number in and produces one number out, using two learnable parameters — a *weight* (the slope) and a *bias* (the intercept). That is all a linear layer is: multiply and add.

```
import torch.nn as nn

model = nn.Linear(1, 1)  # 1 input, 1 output

print(model.weight, model.bias)
```

OUTPUT

```
Parameter containing: tensor([[...]]) Parameter containing: tensor([[...]])
```

PyTorch fills the weight and bias with small random numbers. Right now the model's predictions are essentially random — it has not learned

anything yet. Our job is to nudge those two numbers until the predictions match the data.

WATCH OUT

The model expects input of shape `(batch, features)` — that is why we made `x` into shape `(20, 1)` rather than a flat `(20,)`. Shape discipline starts here and never stops.

Step 3 — the loss function

We need a single number that says "how wrong is the model right now?" For a problem where the output is a continuous value, the standard choice is *mean squared error* (MSE): take the difference between prediction and truth at every point, square it, and average.

```
loss_fn = nn.MSELoss()
```

That is it — one line. When we call `loss_fn(prediction, y)`, it returns a single scalar tensor: the average squared error. A large number means the model is very wrong; zero means it is perfect. The whole goal of training is to drive this number down.

Step 4 — the optimizer

The optimizer is the mechanism that actually changes the model's numbers. We give it the model's parameters and a *learning rate* — a small number that controls how big each adjustment is.

```
optimizer = torch.optim.SGD(  
    model.parameters(), lr=0.1  
)
```


SGD stands for *stochastic gradient descent*. We will explain what that means in Part II — for now, just know that it looks at the gradients (which direction reduces the error) and nudges the parameters a small step in that direction. The learning rate `0.1` says "take modest steps." Too large and the model overshoots; too small and it barely moves.

Step 5 — the training loop

Here is where everything comes together. The loop repeats a simple ritual: make a prediction, measure the error, compute the gradients, adjust the parameters, and reset. Watch:

```
for epoch in range(200):
    pred = model(x)           # ❶ forward pass
    loss = loss_fn(pred, y)   # ❷ measure error

    optimizer.zero_grad()     # ❸ reset gradients
    loss.backward()           # ❹ compute gradients
    optimizer.step()          # ❺ update parameters

    if epoch % 50 == 0:
        print(f"Epoch {epoch:3d} "
              f"Loss: {loss.item():.4f}")
```

OUTPUT

```
Epoch   0  Loss: 8.5483
Epoch  50  Loss: 0.1091
Epoch 100  Loss: 0.0830
Epoch 150  Loss: 0.0804
```

The loss plummets from 8.5 down to 0.08 — the model went from knowing nothing to fitting the data closely, in 200 tiny steps. That is learning.

What just happened

Five lines inside the loop, running two hundred times. Let us name each one, because you will write these same five lines in almost every project for

the rest of this book — and beyond.

- 1 **Forward pass.** Feed $\mathbf{x}$ through the model to get a prediction.
- 2 **Compute the loss.** A single number: how wrong is the prediction?
- 3 **Zero the gradients.** Clear leftover gradients from the previous step so they don't accumulate.
- 4 **Backward pass.** Trace back through every operation and calculate the gradient of the loss with respect to each parameter.
- 5 **Step.** Nudge each parameter a small amount in the direction that reduces the loss.

You do not need to understand *how* gradients work yet — that is Part II. For now, trust that the loop drives the error down, and notice the *shape* of the code: forward, loss, zero, backward, step. It is a rhythm, and it stays the same whether the model is one layer or one hundred.

Did it actually learn the right answer?

Recall that the true relationship was $y = 3x + 1$. The model had no idea — it started with random numbers. Let's see what it found:

```
w = model.weight.item()
b = model.bias.item()
print(f"Learned: y = {w:.2f}x + {b:.2f}")
print(f"Truth:   y = 3.00x + 1.00")
```

OUTPUT

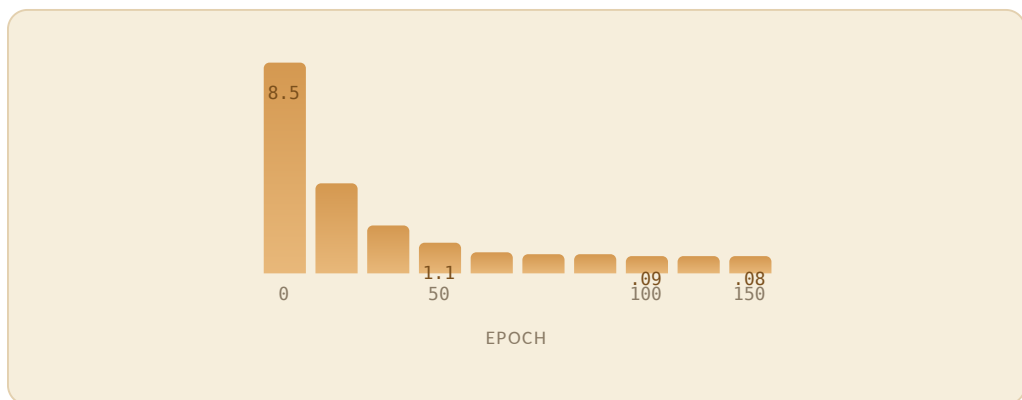
```
Learned: y = 2.97x + 1.07
Truth:   y = 3.00x + 1.00
```

Close enough to feel like magic — and this is the simplest possible model. The weight landed near 3 and the bias near 1, discovered entirely from the

noisy data points. The noise prevents a perfect recovery, and that is exactly what real data looks like.

The loss falling is the proof

If you plotted the loss at every epoch, you would see a curve that drops steeply at first, then flattens. That curve has a name — the *loss curve* — and it is the single most important diagnostic in all of training. A falling loss means the model is learning. A flat loss means it has stalled. A rising loss means something is wrong. You will draw this curve a hundred times.



The loss curve: steep drop at first, then a long, flat tail. This shape is normal and healthy.

The whole thing, in one place

Here is every line together, without interruption. Fifteen lines of real, runnable code, plus a few prints. Copy it, run it, and watch it learn.

```
import torch
import torch.nn as nn

torch.manual_seed(0)

# --- Data ---
x = torch.linspace(0, 2, 20).unsqueeze(1)
y = 3 * x + 1 + 0.3 * torch.randn_like(x)

# --- Model, Loss, Optimizer ---
model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(
    model.parameters(), lr=0.1)

# --- Training loop ---
for epoch in range(200):
    loss = loss_fn(model(x), y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

w = model.weight.item()
b = model.bias.item()
print(f"y ≈ {w:.2f}x + {b:.2f}")
```

TRY IT

Run the complete listing. Then change the true relationship to $y = 5x + 2$ (modify only the data line) and train again. Does the model find the new slope and intercept?

WATCH OUT

If you forget `optimizer.zero_grad()`, the gradients from every previous step pile up, and the model quickly diverges. It is the most common single-line bug in PyTorch training loops.

What comes next

You just saw the entire machine-learning pipeline in miniature. Over the next thirty chapters, we will take it apart piece by piece, understand how each one works, and put it back together with increasing power — until the model is no longer a single line, but a transformer built from the ground up. But the rhythm stays the same: forward, loss, zero, backward, step. Remember that, and every chapter from here has a home.

THE RULE

- 1 Every project follows the same five steps: **Data** → **Model** → **Loss** → **Optimizer** → **Loop**.
- 2 The training loop has five lines: forward, loss, zero_grad, backward, step.
- 3 The loss curve is the proof: falling means learning.
- 4 You do not need to understand every piece today. Seeing the whole once is enough.

END OF CHAPTER 3

Exercises

1. Run the complete 15-line model. Confirm the final output is close to $y \approx 3.00x + 1.00$.
2. Change the data to $y = -2x + 5$ (with noise). Train again. What does the model learn?
3. Try changing the learning rate from **0.1** to **0.001**. Run for 200 epochs. Is the final loss higher or lower? Why?
4. Now try a learning rate of **10.0**. What happens to the loss? (Hint: watch for **nan** or huge numbers.)

5. Remove the line `optimizer.zero_grad()` and train again. Describe what happens to the loss, and explain in one sentence why.
6. Print `model.weight` and `model.bias` *before* training starts. Are they close to the true values? Now print them again after training. What changed?

INTERVIEW-STYLE PROBLEM

An interviewer gives you this prompt: "Walk me through the five-line PyTorch training loop. What does each line do, and what happens if you remove `zero_grad`?" Give a concise, clear answer.

PART I · TENSOR FLUENCY — CHAPTER 5

Indexing & Slicing

*Reach into a tensor and pull out exactly the piece you need
— by position, by range, or by condition.*

Creating tensors is only half the skill. The other half is reaching inside them — selecting a single element, a row, a column, every other value, or every element that satisfies a condition. This chapter gives you the full indexing vocabulary, from the everyday basics to the patterns that show up inside real models. By the end, you will be able to slice any tensor along any axis without hesitation, and you will know when what you get back is a *view* of the original data versus an independent *copy*.

The tensor we'll use

To make every example concrete, we will work with a single 4×5 matrix throughout this chapter. Think of each cell as labelled by its row and column so you can always trace what gets selected.

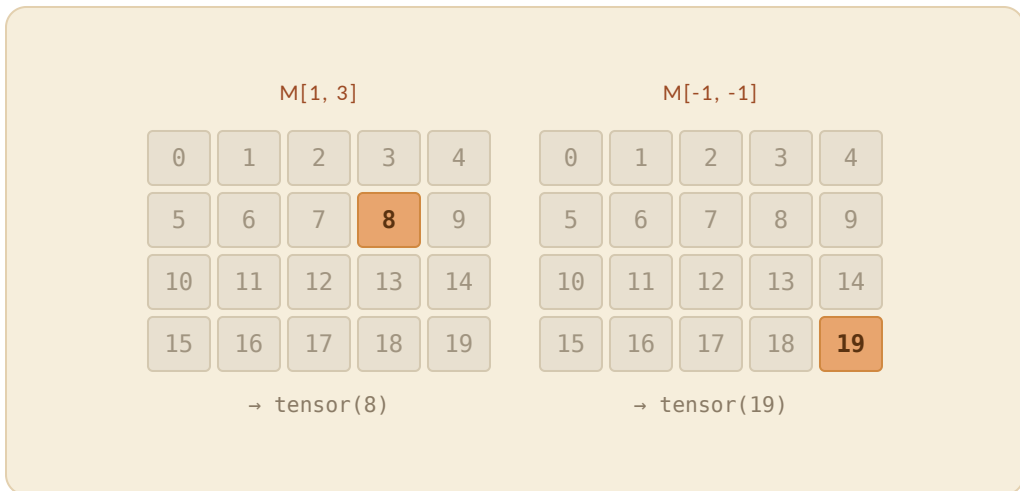
```
m = torch.arange(20).reshape(4, 5)
print(m)
```

OUTPUT

```
tensor([[ 0,  1,  2,  3,  4],
        [ 5,  6,  7,  8,  9],
        [10, 11, 12, 13, 14],
        [15, 16, 17, 18, 19]])
```

Single element

Index with one integer per dimension. Row first, then column — just like coordinates on a grid.

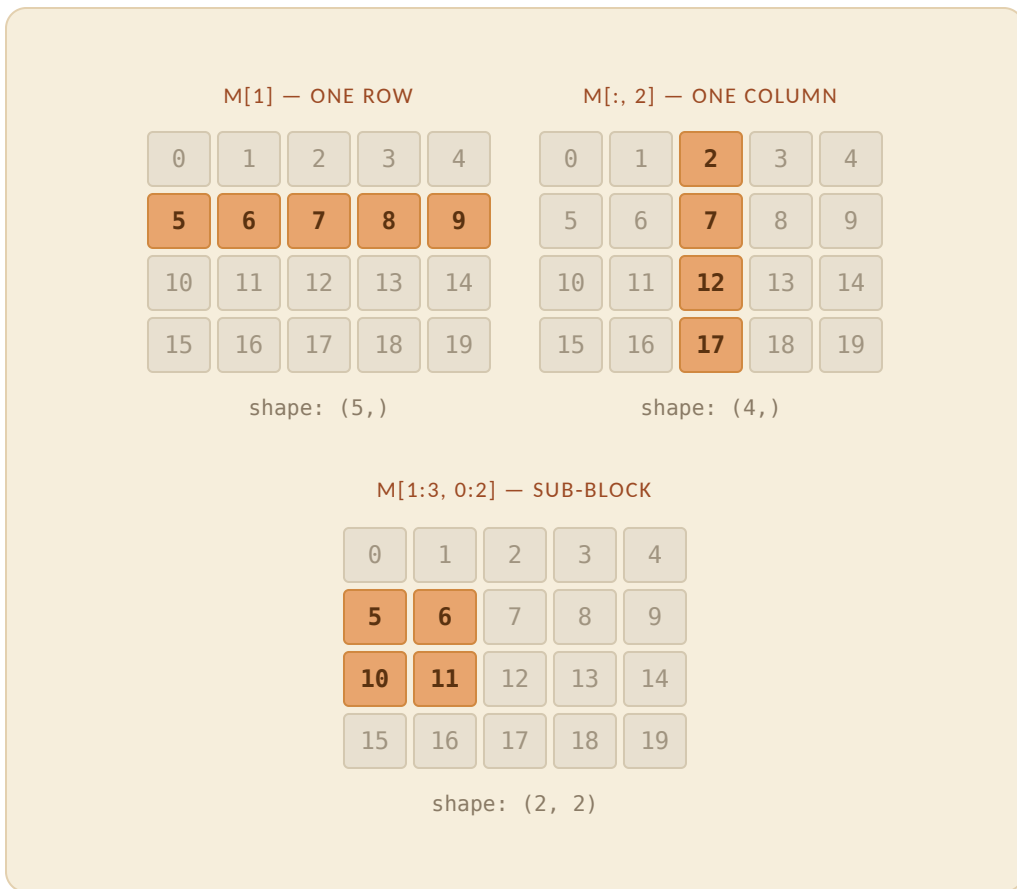


Positive indices count from 0; negative indices count from the end.

```
print(m[1, 3])    # 8
print(m[-1, -1]) # 19 (last row, last col)
```

Row and column slicing

Use the familiar `start:stop` syntax to grab a range. Just as in Python, the start is included and the stop is excluded.



The colon `:` means "all" along that axis. A range `1:3` means indices 1 and 2.

```
print(m[1])           # row 1
print(m[:, 2])        # column 2
print(m[1:3, 0:2])    # rows 1-2, cols 0-1
```

Step slicing

Add a third number — `start:stop:step` — to skip elements. This works exactly like Python list slicing.

```
print(m[::2])          # every other row
print(m[:, ::2])       # every other column
print(m[::-1])         # rows reversed
```

OUTPUT

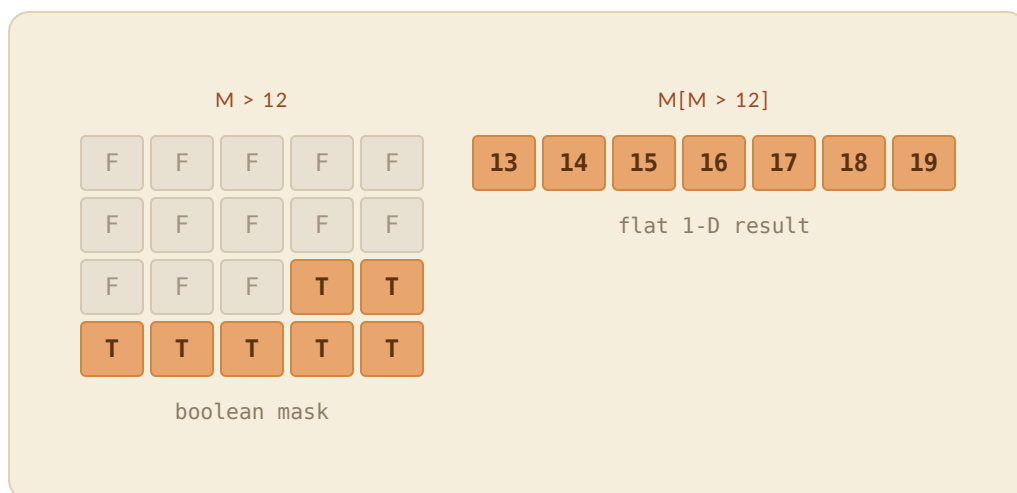
```
tensor([[ 0,  1,  2,  3,  4],
        [10, 11, 12, 13, 14]])
tensor([[ 0,  2,  4],
        [ 5,  7,  9],
        [10, 12, 14],
        [15, 17, 19]])
tensor([[15, 16, 17, 18, 19],
        [10, 11, 12, 13, 14],
        [ 5,  6,  7,  8,  9],
        [ 0,  1,  2,  3,  4]])
```

TRY IT

From our 4×5 matrix, extract every other element from row 2 (i.e. `m[2, ::2]`). Predict the result before running it, then check.

Boolean (mask) indexing

Instead of saying *where* to look, you can say *what condition* to match. A comparison creates a boolean tensor of the same shape, and passing that boolean tensor as an index pulls out every element where the condition is true.



The mask is True where the condition holds. Indexing with it returns a flat 1-D tensor of the matching values.

```
mask = m > 12
print(mask)
print(m[mask])
```

OUTPUT

```
tensor([[False, False, False, False, False],
        [False, False, False, False, False],
        [False, False, False,  True,  True],
        [ True,  True,  True,  True,  True]])
tensor([13, 14, 15, 16, 17, 18, 19])
```

Boolean indexing is how you filter data, zero out values below a threshold, or select only the positive activations in a model. It always returns a *flat* result — the grid structure is lost, because the number of True entries can vary.

WATCH OUT

Boolean indexing always returns a *copy*, not a view. Modifying the result will not change the original tensor. If you want to modify the original in place, assign through the mask: `m[m > 12] = 0`.

Fancy (integer-list) indexing

You can also index with a list (or tensor) of integers to pick specific rows or elements in any order, with repetition if you like.

```
# Pick rows 0, 2, and 3
print(m[[0, 2, 3]])

# Pick specific (row, col) pairs
rows = torch.tensor([0, 1, 3])
cols = torch.tensor([4, 2, 0])
print(m[rows, cols])
```

OUTPUT

```
tensor([[ 0,  1,  2,  3,  4],
        [10, 11, 12, 13, 14],
        [15, 16, 17, 18, 19]])
tensor([ 4,  7, 15])
```

The second pattern — pairing a row tensor with a column tensor — selects the elements at positions (0, 4), (1, 2), and (3, 0). This shows up when you need to gather specific predictions or labels from a batch.

WATCH OUT

Like boolean indexing, fancy indexing with a list or tensor returns a *copy*. Only basic slicing (the `start:stop:step` kind) returns a view.

Views vs copies: when slicing shares memory

This is one of the most important practical details in all of PyTorch. When you slice with the basic `start:stop` syntax, the result is a *view* — it shares the same underlying data. Modify the slice, and the original changes too.

```
row = m[0]          # a view of row 0
row[0] = 99
print(m[0])         # the original changed!
```

OUTPUT

```
tensor([99,  1,  2,  3,  4])
```

This is fast — no data is copied. But it means you must be careful: if you do not want to modify the original, call `.clone()` to make an independent copy.

```
# Reset our tensor
m = torch.arange(20).reshape(4, 5)

safe = m[0].clone() # independent copy
safe[0] = 99
print(m[0])         # still [0, 1, 2, 3, 4]
```

OUTPUT

```
tensor([0, 1, 2, 3, 4])
```

TRY IT

Take a slice `s = m[1:3, 1:4]`. Set `s[:] = 0`. Now print `m` — which cells changed? Explain why, in terms of views.

Assigning through an index

You can write into a tensor through any kind of index — basic, boolean, or fancy. This is how you zero out bad values, clamp activations, or replace specific elements.

```
m = torch.arange(20).reshape(4, 5)

m[0, :] = -1          # fill row 0 with -1
m[m < 0] = 0          # replace negatives
print(m)
```

OUTPUT

```
tensor([[ 0,  0,  0,  0,  0],
        [ 5,  6,  7,  8,  9],
        [10, 11, 12, 13, 14],
        [15, 16, 17, 18, 19]])
```

A quick note on `gather`

You will occasionally encounter `torch.gather`, especially when selecting specific class scores from model outputs. It lets you pick one element per row (or per any dimension) using an index tensor. We will use it in later chapters — for now, just know it exists and that it solves the pattern "from each row, pick the column whose index is stored in another tensor."

```
scores = torch.tensor([[0.1, 0.9, 0.3],
                       [0.8, 0.1, 0.4]])
labels = torch.tensor([[1], [0]])

picked = scores.gather(1, labels)
print(picked)
```

OUTPUT

```
tensor([[0.9000],
        [0.8000]])
```

Row 0 picked column 1 (0.9), and row 1 picked column 0 (0.8). That is exactly the pattern used when computing the loss for each example in a batch.

THE RULE

- 1 Single element: `t[row, col]`. Negative indices count from the end.
- 2 Slice: `t[start:stop:step]`. Colon `:` means "all." Stop is excluded.
- 3 Boolean mask: `t[t > 0]` returns a flat copy of matching elements.
- 4 Fancy index: `t[[0, 2, 3]]` picks specific positions; also a copy.
- 5 Basic slicing returns a **view** (shared memory). Boolean and fancy indexing return a **copy**.
- 6 Use `.clone()` when you need an independent copy from a view.

END OF CHAPTER 5

Exercises

1. Create `torch.arange(20).reshape(4, 5)`. Extract the element at row 2, column 3 using positive indices, then again using negative indices.
2. From the same matrix, extract the 2×3 sub-block in the bottom-right corner. What slice expression did you use?
3. Reverse the columns of the matrix (so column 4 becomes column 0). Write the one-liner.
4. Create a boolean mask that selects all even numbers from the matrix. Use it to extract those numbers into a 1-D tensor.
5. Take a basic slice `s = m[0:2]`. Modify `s[0, 0] = -1`. Does `m` change? Now do the same but use `.clone()`. Does `m` change this time?
6. Create a (3, 4) tensor of random scores and a (3, 1) tensor of integer labels in `[0, 3]`. Use `gather` to pick the score at each label's column.

INTERVIEW-STYLE PROBLEM

An interviewer asks: "In PyTorch, when you do `a = t[0:5]`, does `a` share memory with `t`? What about `a = t[t > 0]`?" Explain the difference in two sentences, and say when you would use `.clone()`.

PART II · HOW LEARNING ACTUALLY HAPPENS — CHAPTER 13

Gradients, Intuitively

A gradient is just a slope — and the slope tells you which way is downhill.

In Chapter 3 you watched a model learn. The loss started high and fell, step by step, until the model's predictions matched the data.

But we skipped over the *how*. How does the model know which direction to adjust its numbers? The answer is a single idea: the *gradient*. This chapter builds that idea from scratch — no calculus required, just the intuition that a slope tells you which way is downhill. Once you have that, the rest of training is just repeating the instruction "step downhill" over and over.

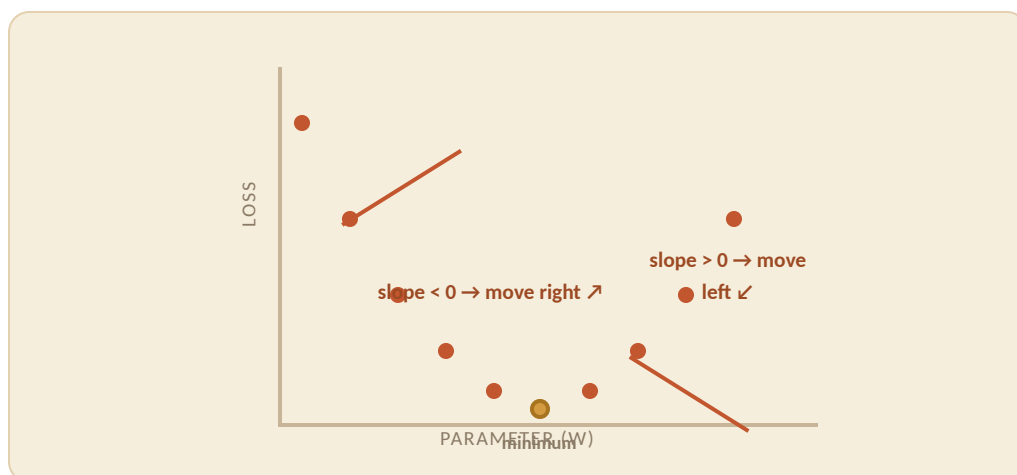
Start with a hill

Imagine you are standing on a hillside in dense fog. You cannot see the bottom, but you can feel the ground under your feet. The ground tilts — it slopes. If you step in the direction the ground tilts downward, you move lower. That is all a gradient is: *the slope of the ground under your feet*. And training a model is nothing more than feeling the slope and stepping downhill, again and again, until you reach the lowest point.

Now replace the hillside with a curve on a graph. The horizontal axis is a parameter — one of the adjustable numbers inside the model. The vertical axis is the loss — the error. The curve shows how the loss changes as you adjust that parameter. Somewhere on this curve there is a lowest point: the value of the parameter that makes the loss as small as possible. The gradient tells you which way to move to get there.

The slope of a curve

Take the simplest possible example: the curve $y = x^2$. At any point on the curve, the slope tells you two things at once: *which direction* is downhill, and *how steep* the hill is.



The curve $y = x^2$. On the left the slope is *negative* (move right). On the right the slope is *positive* (move left). At the bottom the slope is zero — you've arrived.

On the left side of the curve, the slope is *negative* — the ground tilts down to the right. So you step right. On the right side, the slope is *positive* — the ground tilts down to the left. So you step left. At the very bottom, the slope is zero — you are at the minimum. That is the entire logic of gradient descent, in one picture.

The gradient as a number

The slope at any point on a curve is a number. For $y = x^2$, the slope at position x is $2x$. (If you know calculus, that is the derivative. If you don't, just accept it as a formula that gives the slope — we will verify it with code in a moment.) At $x = 3$ the slope is 6: steep and positive. At $x = -2$ the slope is -4 : steep and negative. At $x = 0$ the slope is 0: flat, the bottom.

```
# Verify the slope of  $y = x^2$  at  $x = 3$ 
x = torch.tensor(3.0, requires_grad=True)
y = x ** 2
y.backward()
print(x.grad)      # tensor(6.) — slope =  $2x = 6$ 
```

OUTPUT

```
tensor(6.)
```

PyTorch computed the slope for us — automatically, without us writing any formula for the derivative. That is autograd, and we will explore it fully in the next chapter. For now, the important thing is the *meaning* of the number: a gradient of 6 at $x = 3$ means "the loss is climbing steeply to the right, so step to the left to reduce it."

Stepping downhill

The rule for one step of gradient descent is beautifully simple:

$$\text{new value} = \text{old value} - \text{learning rate} \times \text{gradient}$$

The minus sign is the whole trick: the gradient points *uphill*, so subtracting it moves you *downhill*. The learning rate controls how big each step is.

```
w = 3.0          # starting position
lr = 0.1         # learning rate

for step in range(10):
    grad = 2 * w      # slope of  $w^2$ 
    w = w - lr * grad # step downhill
    print(f"w={w:.4f}  loss={w**2:.4f}")
```

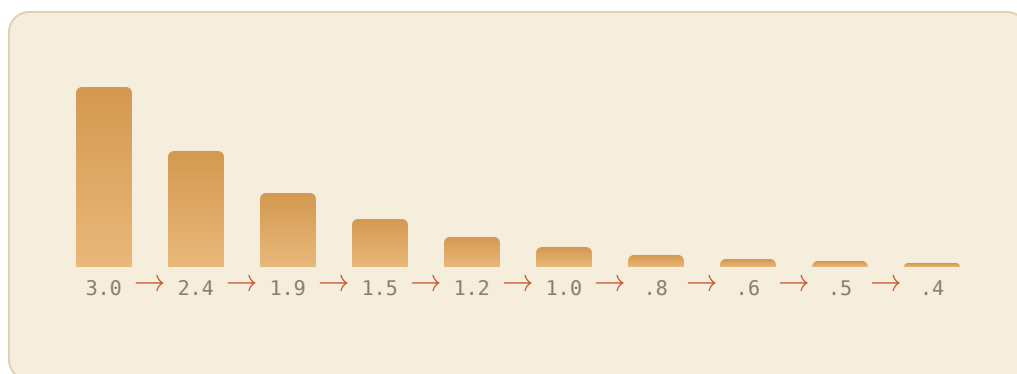
OUTPUT

```

w=2.4000  loss=5.7600
w=1.9200  loss=3.6864
w=1.5360  loss=2.3593
w=1.2288  loss=1.5099
w=0.9830  loss=0.9664
w=0.7864  loss=0.6185
w=0.6291  loss=0.3958
w=0.5033  loss=0.2533
w=0.4027  loss=0.1621
w=0.3221  loss=0.1038

```

Starting at $w = 3$, each step brings w closer to zero — the minimum of w^2 . The loss shrinks from 9.0 down to 0.1 in ten steps. The gradient pointed uphill; the minus sign turned it downhill; the learning rate kept the steps modest. That is gradient descent.



Ten steps of gradient descent on $y = w^2$. The parameter (bottom) and the loss (bar height) both shrink toward zero.

What about many parameters?

A real model has thousands or millions of parameters, not just one. The idea is identical: the gradient is now a *vector* with one number per parameter, and each number tells you the slope of the loss with respect to that one parameter. Updating is the same rule applied to every parameter at once:

```
# Conceptually, for every parameter:  
# param = param - lr * grad_of_loss_wrt_param
```

PyTorch computes the entire vector of gradients for you in a single call to `.backward()`. You never need to write the derivative formulas by hand — the library traces every operation you performed and works the slopes out automatically. We will see exactly how in the next chapter.

The learning rate

The learning rate is the one number you choose, and it matters more than almost anything else in training. Too large, and the steps overshoot the minimum — the loss bounces or explodes. Too small, and the steps are so tiny that training takes forever, or stalls in a shallow dip that is not the real bottom.

```
# Too large – overshoots  
w, lr = 3.0, 1.5  
for _ in range(5):  
    w = w - lr * (2 * w)  
    print(f"w={w:.1f}  loss={w**2:.1f}")
```

OUTPUT

```
w=-6.0  loss=36.0  
w=12.0  loss=144.0  
w=-24.0  loss=576.0  
w=48.0  loss=2304.0  
w=-96.0  loss=9216.0
```

With a learning rate of 1.5, the steps are so large that `w` flies past the minimum and the loss *explodes*. This is the most common cause of `nan` or `inf` in training — and the first thing to reduce when it happens.

WATCH OUT

If the loss suddenly jumps to `nan` or `inf` during training, the learning rate is almost certainly too high. Reduce it by a factor of 10 and try again.

Connecting back to Chapter 3

Now you can read the training loop from Chapter 3 with new eyes. The five lines inside the loop are exactly this chapter's story:

```
pred = model(x)           # ❶ compute prediction
loss = loss_fn(pred, y)    # ❷ measure error
optimizer.zero_grad()      # ❸ clear old slopes
loss.backward()            # ❹ compute the gradient
optimizer.step()           # ❺ step downhill
```

Line 4 computes the gradient — the slope of the loss with respect to every parameter. Line 5 subtracts `lr × gradient` from every parameter. That is gradient descent, running inside a loop. The model is feeling the slope and stepping downhill, two hundred times.

TRY IT

Run the 10-step gradient descent on `w2` from this chapter. Then change the starting value to `w = -5.0`. Does it still find the minimum? Now try `lr = 0.01` with 10 steps — is it close to zero? How many steps does it need?

THE RULE

- 1 A gradient is the slope of the loss with respect to a parameter.
- 2 A **negative** gradient means "the loss decreases if you increase the parameter." A **positive** gradient means "the loss decreases if you decrease it."
- 3 Gradient descent: **new = old - lr × gradient**. The minus sign turns "uphill" into "downhill."
- 4 The learning rate controls step size. Too big → overshoot. Too small → stall.
- 5 For many parameters, the gradient is a vector — one slope per parameter. PyTorch computes the whole vector with **.backward()**.

END OF CHAPTER 13

Exercises

1. Use PyTorch's autograd to compute the gradient of $y = x^2$ at $x = 5$. What do you get? Confirm it equals $2x$.
2. Run gradient descent on $y = (x - 4)^2$ starting from $x = 0$ with $lr = 0.1$ for 30 steps. What value does x converge to? Why?
3. Try gradient descent on $y = x^2$ with $lr = 0.01, 0.1, 0.5$, and 1.1 . Run each for 20 steps. Which converges fastest? Which diverges?
4. Write a gradient descent loop for two parameters at once: minimize $y = a^2 + b^2$ starting from $a = 3, b = 4$. Print both values at each step. Do they both approach zero?
5. In one sentence, explain why the training loop calls **zero_grad()** before **backward()**. What would happen if you skipped it?

INTERVIEW-STYLE PROBLEM

An interviewer asks: "In plain language, what is a gradient and how does a neural network use it to learn?" Answer in three sentences without using any formulas.

PART III · BUILDING MODELS FROM BLOCKS — CHAPTER 19

nn.Module

The one class every PyTorch model inherits from — and the pattern you will write a hundred times.

In the last chapter you built a model from raw tensors: you created each parameter by hand, wrote the forward pass as arithmetic, wrapped the update in `no_grad`, and zeroed the gradients yourself. It worked — but it was fragile, verbose, and impossible to scale. `nn.Module` is PyTorch's answer to all of those problems. It is a base class that gives your model parameter tracking, a clean forward method, device management, saving and loading, and the ability to nest smaller modules into bigger ones. Every model you build from this point forward — from a single linear layer to a billion-parameter transformer — will be a subclass of `nn.Module`.

The pattern

Every custom model has the same two-method shape: `__init__` to set up the parts, and `forward` to say how data flows through them.

```
import torch
import torch.nn as nn

class TinyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(1, 1)

    def forward(self, x):
        return self.linear(x)
```

That is a complete, trainable model. Let us walk through each part.

class **TinyModel(nn.Module)**

Your model inherits from `nn.Module`. This gives it parameter tracking, `.to(device)`, saving/loading, and more — for free.

`__init__`

Create the layers and store them as attributes.

`super().__init__()` must be the first line — it sets up the internal plumbing.

forward

Define how data flows through the layers. This is called when you write `model(x)`. Never call `forward(x)` directly.

WATCH OUT

Always call `model(x)`, not `model.forward(x)`. Calling the model directly runs extra hooks (for things like dropout and batch normalization) that `forward` alone would skip. This matters the moment your model includes any layer with train/eval behavior.

Using the model

Once defined, you instantiate it, pass data through it, and train it with the same loop you already know.

```
torch.manual_seed(0)

# Data
x = torch.linspace(0, 2, 20).unsqueeze(1)
y = 3 * x + 1 + 0.3 * torch.randn_like(x)

# Model, loss, optimizer
model = TinyModel()
loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(
    model.parameters(), lr=0.1)

# Training loop — unchanged
for epoch in range(200):
    loss = loss_fn(model(x), y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

w = model.linear.weight.item()
b = model.linear.bias.item()
print(f"y ≈ {w:.2f}x + {b:.2f}")
```

OUTPUT

```
y ≈ 2.97x + 1.07
```

The same result as Chapter 3 and Chapter 18 — but the model is now a proper class, the optimizer handles every parameter at once, and nothing is fragile.

.parameters(): what the optimizer sees

When you write `model.parameters()`, `nn.Module` walks through every attribute you assigned in `__init__` and collects every learnable tensor — recursively, through nested sub-modules. That is how the optimizer knows what to update.

```
for name, param in model.named_parameters():  
    print(name, param.shape)
```

OUTPUT

```
linear.weight torch.Size([1, 1])  
linear.bias torch.Size([1])
```

Two parameters, automatically discovered. If you add more layers in `__init__`, their parameters appear here with no extra work. This is the real power of `nn.Module`: it makes parameter management scale.

TRY IT

Count the total number of learnable parameters: `sum(p.numel() for p in model.parameters())`. For our tiny model, what is the answer?

A two-layer model

The pattern extends naturally. To build a model with two layers and a ReLU in between, just add more lines in `__init__` and chain them in `forward`:

```

class TwoLayer(nn.Module):
    def __init__(self, in_dim, hidden, out_dim):
        super().__init__()
        self.layer1 = nn.Linear(in_dim, hidden)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(hidden, out_dim)

    def forward(self, x):
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        return x

model = TwoLayer(1, 16, 1)
print(sum(p.numel() for p in model.parameters()))

```

OUTPUT

49

Forty-nine parameters (16 weights + 16 biases in layer 1, 16 weights + 1 bias in layer 2) — and `model.parameters()` found all of them automatically. The training loop does not change at all.

Printing the model

PyTorch gives you a free `print` that shows the architecture at a glance:

```
print(model)
```

OUTPUT

```

TwoLayer(
  (layer1): Linear(in_features=1, out_features=16, bias=True)
  (relu): ReLU()
  (layer2): Linear(in_features=16, out_features=1, bias=True)
)

```

This printout comes from `nn.Module` for free and updates itself as you add or remove layers. It is the fastest way to sanity-check a model's structure.

Moving to a device

Because `nn.Module` tracks all parameters, moving the entire model to a device is a single call:

```
device = "cuda" if torch.cuda.is_available() \
        else "cpu"
model = model.to(device)    # every parameter moves
```

Every parameter inside the model — including inside nested sub-modules — moves at once. No parameter-by-parameter loop, no risk of forgetting one. This is another reason the raw-tensor approach from Chapter 18 does not scale.

WATCH OUT

After `model.to(device)`, your **data** must also be on the same device: `x = x.to(device)`. Mixing CPU model and GPU data (or vice versa) is a runtime error — and it is one of the most common mistakes when first using a GPU.

The contract

Every `nn.Module` subclass follows the same contract:

THE RULE

- 1 Inherit from `nn.Module`. Call `super().__init__()` first in `__init__`.
- 2 Create layers and sub-modules as **attributes** in `__init__` so they are registered.
- 3 Define the data flow in `forward`. Call the model with `model(x)`, never `model.forward(x)`.
- 4 `.parameters()` collects every learnable tensor — recursively through all sub-modules.
- 5 `.to(device)` moves every parameter at once.
- 6 Printing the model shows its full architecture for free.

END OF CHAPTER 19

Exercises

1. Write a `ThreeLayer` model with two hidden layers of size 32 and 16, ReLU between each. Print the model and count its parameters.
2. Train the `TwoLayer` model on the $y = 3x + 1$ data from this chapter using Adam. Does it converge? Compare the final loss to the single-layer version.
3. Call `model.named_parameters()` on your `ThreeLayer` model. What names does PyTorch assign to each weight and bias?
4. Try storing a layer as a local variable in `forward` instead of an attribute in `__init__`. Does `model.parameters()` find it? Why or why not?
5. Move a model to `"cpu"` (if you have no GPU, this is a no-op). Print `next(model.parameters()).device` to confirm. Then try passing a tensor on `"cpu"` through a model on `"cuda"` (or vice versa) — what error do you get?

INTERVIEW-STYLE PROBLEM

An interviewer asks: "What does `nn.Module` give you that raw tensors don't?" List three concrete things in your answer, and explain why you should call `model(x)` instead of `model.forward(x)`.

PART IV · FEEDING DATA — CHAPTER 23

Dataset

The contract between your data and PyTorch — two methods that unlock everything the DataLoader can do.

Up to now, every model in this book has trained on tensors we typed by hand — twenty floating-point numbers, a `linspace`, some random noise. Real projects have thousands or millions of samples stored in CSV files, image folders, databases, or in-memory arrays. The `Dataset` class is PyTorch's answer to that gap: it wraps *any* data source behind a simple two-method interface, and once wrapped, the entire `DataLoader` machinery — batching, shuffling, parallel loading — works automatically. This chapter teaches you that interface, shows you the built-in shortcut for tensor data, and then walks you through writing a custom `Dataset` from scratch.

The contract: two methods

A PyTorch `Dataset` is any Python object that implements two methods:

`__len__(self)`

Return the total number of samples. Called by `len(dataset)`.

`__getitem__(self, idx)`

Return the sample at position `idx`. Called by `dataset[idx]`.

That is the entire contract. If your class has these two methods, PyTorch's `DataLoader` can iterate over it, batch it, shuffle it, and load it in parallel. Everything in the next chapter depends on this.

The quick way: `TensorDataset`

When your data is already a pair of tensors — features and labels — you do not need to write a class at all. `TensorDataset` wraps them for you.

```
from torch.utils.data import TensorDataset

x = torch.randn(100, 5)    # 100 samples, 5 features
y = torch.randint(0, 3, (100,)) # 100 labels (3 classes)

ds = TensorDataset(x, y)

print(len(ds))            # 100
print(ds[0])              # (tensor([...]), tensor(2))
print(ds[0][0].shape)     # torch.Size([5])
```

OUTPUT

```
100
(tensor([ 1.5410, -0.2934, -2.1788,  0.5684, -1.0845]), tensor(1))
torch.Size([5])
```

Each call to `ds[i]` returns a tuple — one element per tensor you passed in. For a simple features-and-labels setup, this is all you need. But the moment your data lives in files, or requires loading on the fly, you need a custom Dataset.

TRY IT

Create a `TensorDataset` with 50 samples of 10 features and 50 labels. Index into it at positions 0, 25, and 49. Print the feature shape and the label for each.

Writing a custom Dataset

A custom Dataset is a class that inherits from `torch.utils.data.Dataset` and implements `__len__` and `__getitem__`. Here is one that wraps a simple in-memory table — imagine it came from a CSV file.

```
from torch.utils.data import Dataset

class HouseDataset(Dataset):
    def __init__(self, features, prices):
        self.features = features  # (N, D) tensor
        self.prices = prices     # (N,) tensor

    def __len__(self):
        return len(self.features)

    def __getitem__(self, idx):
        return self.features[idx], self.prices[idx]
```

```
# Create some fake data
feats = torch.randn(200, 3)  # 200 houses, 3 features
prices = torch.randn(200)    # 200 prices

ds = HouseDataset(feats, prices)
print(len(ds))               # 200
print(ds[0][0].shape)        # (3,) — one house
```

OUTPUT

```
200
torch.Size([3])
```

The pattern is always the same: store the data in `__init__`, return the length in `__len__`, return one sample in `__getitem__`. What changes between datasets is only *where* the data comes from and *how* you load it.

A Dataset from a CSV file

In a real project, data often lives in a CSV. The Dataset loads it once in `__init__` and serves one row at a time from `__getitem__`.

```
import csv

class CSVDataset(Dataset):
    def __init__(self, filepath):
        with open(filepath) as f:
            reader = csv.reader(f)
            next(reader) # skip header
            rows = [row for row in reader]

            self.features = torch.tensor(
                [[float(v) for v in row[:-1]]
                 for row in rows])
            self.labels = torch.tensor(
                [int(row[-1]) for row in rows])

    def __len__(self):
        return len(self.features)

    def __getitem__(self, idx):
        return self.features[idx], self.labels[idx]
```

The CSV is read once, converted to tensors, and stored. After that, indexing is instant. For very large files that do not fit in memory, you can store only the file paths in `__init__` and read each sample lazily in `__getitem__` — a pattern we will see with images.

WATCH OUT

`__getitem__` should return **tensors**, not Python lists or NumPy arrays. The DataLoader expects tensors so it can stack them into batches. If you return raw Python objects, you will get cryptic collation errors later.

A Dataset for images (lazy loading)

Images are too large to load all at once, so the standard pattern stores only the file *paths* in `__init__` and loads each image on demand in `__getitem__`. This is how every major image dataset works.

```
from pathlib import Path
# from PIL import Image # uncomment with real images

class ImageDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.paths = sorted(
            Path(root_dir).glob("*/*.jpg"))
        self.classes = sorted(
            {p.parent.name for p in self.paths})
        self.class_to_idx = {
            c: i for i, c in enumerate(self.classes)}
        self.transform = transform

    def __len__(self):
        return len(self.paths)

    def __getitem__(self, idx):
        path = self.paths[idx]
        # img = Image.open(path).convert("RGB")
        img = torch.randn(3, 32, 32) # placeholder
        label = self.class_to_idx[path.parent.name]
        if self.transform:
            img = self.transform(img)
        return img, label
```

The folder structure encodes the labels: `root/cat/001.jpg`, `root/dog/001.jpg`. The `transform` argument is a hook for preprocessing — resizing, normalizing, augmenting — which we will cover in Chapter 25.

TRY IT

Write a custom Dataset that wraps two NumPy arrays (features and labels). Convert them to tensors inside `__init__`. Verify that `len()` and `dataset[0]` work correctly.

Built-in datasets

PyTorch's `torchvision`, `torchaudio`, and `torchtext` libraries include ready-made datasets for common benchmarks. They follow the same `__len__`/`__getitem__` contract.

```
from torchvision import datasets

# Downloads MNIST to ./data on first run
mnist = datasets.MNIST(
    root="./data",
    train=True,
    download=True,
)

print(len(mnist))          # 60000
img, label = mnist[0]
print(type(img), label)    # PIL Image, 5
```

These are real Datasets — you can pass them straight to a `DataLoader`. We will use MNIST and other built-in datasets throughout Parts V, VI, and VII.

What the `DataLoader` expects

This chapter focused on the Dataset side. In the next chapter, we will plug our datasets into a `DataLoader`, which handles batching, shuffling, and parallel loading. But the `DataLoader`'s only requirement is this: *give*

me an object with `__len__` and `__getitem__` that returns tensors. That is the contract, and now you can fulfill it for any data source.

THE RULE

- 1 A Dataset implements two methods: `__len__` (how many samples) and `__getitem__` (one sample by index).
- 2 `TensorDataset` wraps existing tensors — no class needed.
- 3 For custom data: inherit from `Dataset`, load data in `__init__`, return one sample in `__getitem__`.
- 4 For large data (images): store `paths` in `init`, load `lazily` in `getitem`.
- 5 `__getitem__` should return **tensors**. The `DataLoader` stacks them into batches.
- 6 Built-in datasets (`torchvision.datasets`) follow the same contract and work out of the box.

END OF CHAPTER 23

Exercises

1. Create a `TensorDataset` with 1000 samples of shape (8,) and 1000 integer labels in [0, 4]. Print `len()` and the shape of the first sample's features.
2. Write a custom `SineDataset` that generates x values from 0 to 2π and $y = \sin(x)$. The dataset should have `n_points` samples. Verify by indexing into it.
3. Modify the `CSVDataset` to also store the column names from the CSV header. Add a `.columns` attribute that returns them.
4. Write an `ImageDataset` for a folder structure `root/class/image.jpg`. You do not need real images — use `torch.randn(3, 32, 32)` as a placeholder. Verify that the labels are correct integers.

5. What happens if `__getitem__` returns a Python list instead of a tensor? Create a Dataset that does this and try to wrap it in a `DataLoader(ds, batch_size=4)`. What error do you get?

INTERVIEW-STYLE PROBLEM

An interviewer asks: "What two methods must a PyTorch Dataset implement, and why does the DataLoader need them?" Answer in three sentences. Then they ask: "When would you load data lazily in `__getitem__` instead of all at once in `__init__`?" Give an example.

PART V · THE TRAINING LOOP, MASTERED — CHAPTER 27

Anatomy of a Training Loop

The six lines that make a model learn — understood so deeply that you could write them on a whiteboard from memory.

Every chapter in this book has been building toward this one. You know how tensors work. You know how gradients and autograd compute the slope. You know how loss functions measure error and how optimizers step downhill. You know how Datasets and DataLoaders serve data in batches. Now we put it all together into the single most important pattern in deep learning: the *training loop*. This chapter presents the canonical loop line by line, explains why each line exists and what breaks if you remove it, and gives you a complete, runnable example that you can copy and adapt for any project.

The canonical loop

Here it is — the shape you will write hundreds of times. Memorize it like a phone number.

```
for epoch in range(num_epochs):          # ❶
    for batch_x, batch_y in train_loader: # ❷
        pred = model(batch_x)             # ❸
        loss = loss_fn(pred, batch_y)     # ❹

        optimizer.zero_grad()             # ❺
        loss.backward()                   # ❻
        optimizer.step()                   # ❼
```

Seven lines. Two loops and five operations. Every supervised training run in PyTorch follows this exact structure, whether it is a two-parameter linear model or a billion-parameter transformer. Let us walk through each line.

Line by line

1 `for epoch in range(num_epochs):`

One **epoch** is a complete pass through the entire training set. Repeat for as many epochs as it takes for the loss to converge.

2 `for batch_x, batch_y in train_loader:`

Within each epoch, the `DataLoader` yields one **batch** at a time. Each batch is a mini-subset of the training data, typically 16–128 samples.

3 `pred = model(batch_x)`

The **forward pass**. Data flows through the model and produces predictions. The computation graph is built here.

4 `loss = loss_fn(pred, batch_y)`

Measure the error. Compare predictions to the true labels. MSE for regression, `CrossEntropy` for classification.

5 `optimizer.zero_grad()`

Clear old gradients. Without this, gradients accumulate from the previous batch and the update is wrong.

6

`loss.backward()`

Backward pass. Autograd walks the computation graph in reverse and fills in `.grad` for every parameter.

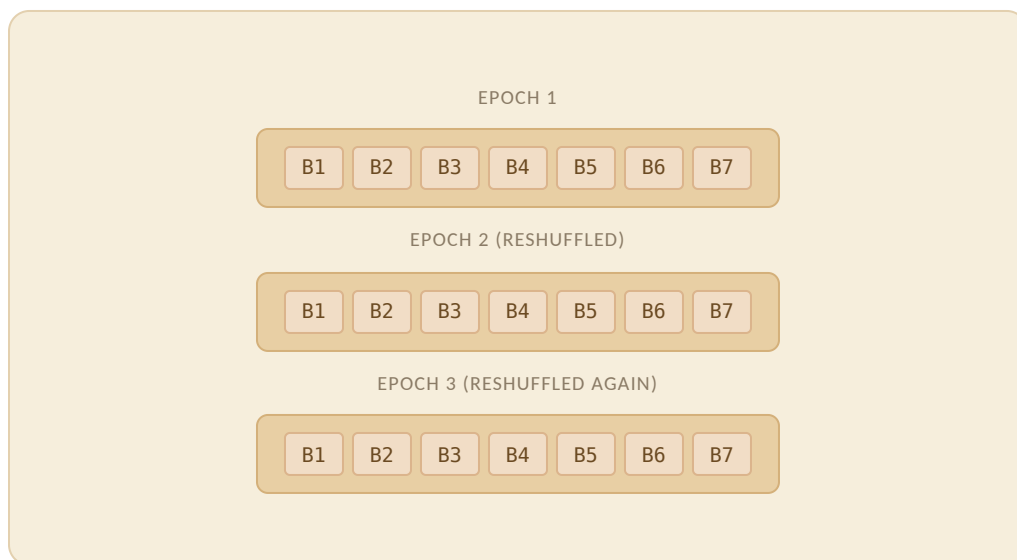
7

`optimizer.step()`

Update parameters. The optimizer reads the gradients and adjusts every parameter by `param -= lr × grad` (or the Adam variant).

Epochs vs batches

This distinction confuses many beginners, so it is worth making visual.



Three epochs, each with 7 batches. Every sample is seen once per epoch, but in a different order each time (if `shuffle=True`).

An epoch is one full pass through the data. A batch is one step within that pass. If you have 1000 samples and a batch size of 32, each epoch has $\lceil 1000/32 \rceil = 32$ batches. The model updates its parameters once per

batch — so one epoch means 32 updates. Ten epochs means 320 updates.

A complete, runnable example

Here is the full loop, from raw data to a trained model, with every import and every line shown. This is the template you will copy for every project.

```

import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

# — Data —————
torch.manual_seed(0)
x = torch.randn(500, 10)
w_true = torch.randn(10, 1)
y = x @ w_true + 0.5 * torch.randn(500, 1)

ds = TensorDataset(x, y)
loader = DataLoader(ds, batch_size=32, shuffle=True)

# — Model, loss, optimizer —————
model = nn.Linear(10, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.Adam(model.parameters(), lr=1e-2)

# — Training loop —————
losses = []
for epoch in range(30):
    epoch_loss = 0.0
    for bx, by in loader:
        pred = model(bx)
        loss = loss_fn(pred, by)

        opt.zero_grad()
        loss.backward()
        opt.step()

    epoch_loss += loss.item() * bx.size(0)

avg_loss = epoch_loss / len(ds)
losses.append(avg_loss)
if epoch % 5 == 0:
    print(f"Epoch {epoch:2d}  Loss: {avg_loss:.4f}")

```

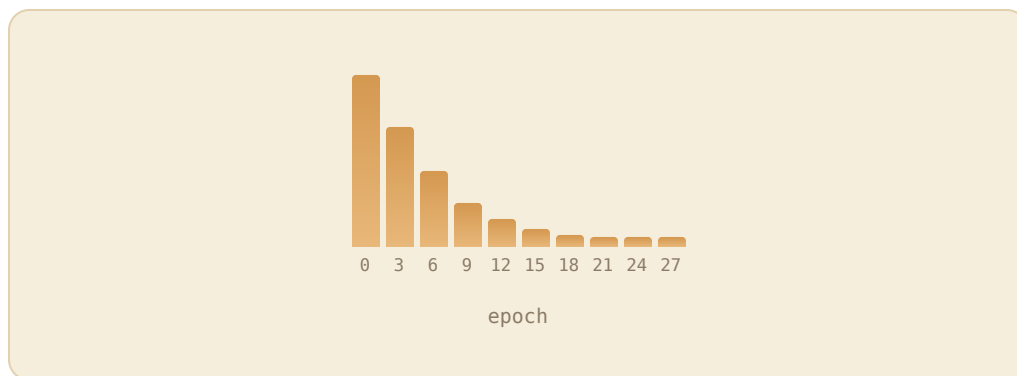
OUTPUT

```
Epoch 0  Loss: 4.3217
Epoch 5  Loss: 0.3854
Epoch 10 Loss: 0.2578
Epoch 15 Loss: 0.2497
Epoch 20 Loss: 0.2491
Epoch 25 Loss: 0.2489
```

The loss drops from 4.3 to 0.25 in 30 epochs — the model has learned the linear relationship. Notice the pattern for computing average epoch loss: `accumulate loss.item() * batch_size`, then divide by total samples. This gives the true epoch average even when the last batch is smaller.

Tracking the loss curve

Appending each epoch's loss to a list and plotting it later is the simplest form of training monitoring. A healthy loss curve drops steeply at first and then flattens out. An unhealthy one oscillates wildly, explodes to infinity, or never drops at all.



A healthy loss curve: steep descent early, then a plateau. The model has converged by epoch 15.

TRY IT

Run the complete example. After training, plot the loss curve: `import matplotlib.pyplot as plt; plt.plot(losses); plt.show()`. Does it look like the bar chart above? Try changing the learning rate to `1e-1` and `1e-4` — how does the curve change?

What breaks if you remove a line?

Each line in the loop exists for a reason. Here is what goes wrong if you remove it — understanding these failures is as important as understanding the correct loop.

WATCH OUT — FIVE WAYS TO BREAK THE LOOP

Skip `zero_grad()` : Gradients accumulate from previous batches. The updates grow larger each step and the model diverges.

Skip `backward()` : No gradients are computed. `step()` has nothing to work with — parameters never change.

Skip `step()` : Gradients are computed but never applied. The model stays at its initial random weights forever.

Swap `backward()` and `step()` : Parameters are updated with stale gradients from the previous batch. The model learns, but poorly and erratically.

Call `model.forward(x)` instead of `model(x)` : Hooks for Dropout and BatchNorm are skipped. The model may behave differently in training vs evaluation without you knowing.

The order, as a mantra

If you remember nothing else from this chapter, remember this order:

predict → loss → zero → backward → step

Five words, always in this order, always inside the batch loop. Every project, every architecture, every dataset. The loop is the heartbeat; this is the rhythm.

Adding validation

A training loop alone tells you how well the model fits the training data — but not how well it generalizes. To check generalization, evaluate on the validation set after each epoch. We will cover this fully in the next chapter, but here is the shape:

```
for epoch in range(num_epochs):
    # - Training -----
    model.train()
    for bx, by in train_loader:
        pred = model(bx)
        loss = loss_fn(pred, by)
        opt.zero_grad()
        loss.backward()
        opt.step()

    # - Validation -----
    model.eval()
    with torch.no_grad():
        val_loss = 0.0
        for bx, by in val_loader:
            pred = model(bx)
            val_loss += loss_fn(pred, by).item()
    print(f"Val loss: {val_loss / len(val_loader):.4f}")
```

Two additions: `model.train()/model.eval()` to toggle Dropout and BatchNorm behavior, and `torch.no_grad()` to skip gradient

computation during evaluation. We will explain both in detail in Chapter 28.

THE RULE

- 1 The training loop has two nested loops: **epochs** (outer) and **batches** (inner).
- 2 Inside the batch loop: **predict** → **loss** → **zero_grad** → **backward** → **step**. Always this order.
- 3 Track the average epoch loss to monitor convergence.
- 4 A healthy loss curve drops steeply then plateaus. Exploding or flat curves mean something is wrong.
- 5 Every line in the loop exists for a reason. Removing or reordering any one breaks training.
- 6 Add a validation loop after each training epoch to monitor generalization.

————— END OF CHAPTER 27 —————

Exercises

1. Type the canonical loop from memory on a blank page — no peeking. Then compare to the chapter. Which line did you forget or misorder?
2. Run the complete example from this chapter. Change the number of epochs to 100. Does the loss keep dropping, or does it plateau? At which epoch?
3. Remove `optimizer.zero_grad()` from the loop and train for 20 epochs. Print the loss each epoch. What happens? Put it back and confirm the fix.
4. Swap the order of `backward()` and `step()`. Does the model still learn? Compare the final loss to the correct version.

5. Modify the example to use a classification setup: 500 samples, 10 features, 3 classes. Use `CrossEntropyLoss` and `argmax` to compute accuracy after each epoch.
6. Add a validation split (80/20) and a validation loop that prints validation loss alongside training loss every 5 epochs. Does the validation loss track the training loss?
7. Change the batch size from 32 to 1 (pure SGD) and to 500 (full-batch GD). How does the loss curve change? Which converges fastest?

INTERVIEW-STYLE PROBLEM

An interviewer says: "Write a PyTorch training loop on the whiteboard. Explain each line." Do it — all seven lines, in order, with a one-sentence explanation for each.

PART VI · EVALUATION, DEBUGGING & SEEING INSIDE — CHAPTER 34

Debugging PyTorch

Every model breaks before it works. This is the systematic craft of finding out why — reading the error, isolating the line, and fixing it fast.

The difference between a beginner and a practitioner is not that the practitioner writes bug-free code — it is that they debug faster. Nearly every PyTorch error you will meet falls into a handful of families: shape mismatches, device and dtype mismatches, NaNs and exploding losses, and autograd gotchas. Once you can recognize each family on sight and know its standard fix, a red wall of traceback stops being frightening and becomes a set of directions. This chapter gives you that recognition, plus a repeatable workflow you can run every time something breaks.

How to read a traceback

A Python traceback looks intimidating because it is long, but it has a strict structure. Read it *bottom to top*: the last line names the error, and the lines above trace the calls that led there, most recent last. Your job is to find the deepest line that lives in *your* code.

```
Traceback (most recent call last):
  File "train.py", line 42, in <module>
    out = model(x)
  File "model.py", line 18, in forward
    x = self.fc(x)
RuntimeError: mat1 and mat2 shapes cannot be
multiplied (32x784 and 256x10)
```

Read from the bottom: the error is a shape mismatch in a matrix multiply. Read up one meaningful step: it happened at `self.fc(x)` on line 18 of your `forward`. You now know *what* (a matmul shape error) and *where* (the `fc` layer). The numbers (32x784) and (256x10) are the two operands — that is your next clue, and it is the subject of the next section.

TRY IT

Deliberately break a model: give an `nn.Linear(256, 10)` an input of shape `(32, 784)` and run it. Read the traceback you get. Practice locating the error type on the bottom line and your own offending line above it — this is a muscle you want trained before you need it.

Shape bugs: the number-one problem

Shape mismatches cause more PyTorch errors than everything else combined. The message `mat1 and mat2 shapes cannot be multiplied (32x784 and 256x10)` means a `Linear` layer expecting 256 input features received 784. The fix is almost always to correct an `in_features`, add a `flatten`, or fix an earlier `reshape`.

The single most effective debugging technique in all of PyTorch is embarrassingly simple: **print shapes everywhere**. Drop `print`

statements through your `forward` method and watch where the shape stops being what you expect.

```
def forward(self, x):
    print("in ", x.shape)
    x = self.conv(x)
    print("conv", x.shape)
    x = x.flatten(1)
    print("flat", x.shape)
    return self.fc(x)
```

OUTPUT

```
in torch.Size([32, 3, 32, 32])
conv torch.Size([32, 16, 32, 32])
flat torch.Size([32, 16384]) ← fc expected 256!
```

The print output pinpoints the break: after flattening, each sample is 16 384 features wide, but your `fc` layer was built for 256. Now the fix is obvious — set the linear layer's `in_features` to 16 384, or add pooling before it. For a permanent guard rather than a temporary print, use an `assert` that documents your assumption and fails loudly when violated:

```
def forward(self, x):
    assert x.dim() == 4, \
        f"want (B,C,H,W), got {tuple(x.shape)}"
    return self.net(x)
```

See every shape at once: `torchinfo`

Sprinkling prints works, but for a whole model there is a better tool. `torchinfo` runs a dummy forward pass and prints a table of every layer with its output shape and parameter count — a complete shape map before you train a single step.

```

from torchinfo import summary

model = MyModel()
summary(model, input_size=(1, 3, 32, 32))

```

OUTPUT (ABRIDGED)

Layer (type)	Output Shape	Param #
Conv2d-1	[1, 16, 32, 32]	448
ReLU-2	[1, 16, 32, 32]	0
Flatten-3	[1, 16384]	0
Linear-4	[1, 10]	163,850
Total params: 164,298		

The `input_size` includes the batch dimension, so `(1, 3, 32, 32)` is one 3-channel 32×32 image. Reading down the Output Shape column, you can catch a mismatch — like that 16 384-wide flatten feeding a layer built for 256 — *before* the traceback ever appears. Making `summary` the first thing you run on any new model saves hours.

Device and dtype mismatches

Two closely related errors show up constantly once you move to a GPU or work with integer labels. Both have one-line fixes once you recognize them.

Error message

Cause & fix

**Expected all
tensors to be on
the same device**

A tensor is on CPU while the model is on GPU (or vice versa). Move both: `x = x.to(device)` and `model.to(device)`. (Chapter 30.)

**expected scalar
type Long but
found Float**

Class labels for `cross_entropy` must be integers. Cast them: `y.long()`. Targets index classes; they are not floats.

**expected scalar
type Float but
found Double**

NumPy arrays arrive as float64 (Double). Cast to float32 on conversion: `torch.tensor(a, dtype=torch.float32)`.

The device error deserves a mental model rather than memorization: *every* tensor involved in an operation must live on the same device. When one strays, PyTorch refuses. The habitual fix is to define `device` once and route every model and batch through `.to(device)`:

```
device = "cuda" if torch.cuda.is_available() \
    else "cpu"
model = model.to(device)

for x, y in loader:
    x, y = x.to(device), y.to(device)
    # ... now everything matches ...
```

NaNs and exploding loss

Sometimes nothing throws an error, but your loss prints as `nan` or rockets to `inf`. This is training silently diverging, and it has a short list of usual suspects: a learning rate that is too high, unnormalized inputs, a `log(0)` or division by zero in a custom loss, or a corrupted sample in the data. Detecting it is one line:

```
for step, (x, y) in enumerate(loader):
    loss = criterion(model(x), y)
    if torch.isnan(loss) or torch.isinf(loss):
        print("bad loss at step", step)
        break
    loss.backward()
```

When a NaN appears deep inside the backward pass rather than the loss itself, PyTorch can point you at the exact operation that produced it. Wrap the step in `detect_anomaly` and it will raise at the offending op with a traceback:

```
with torch.autograd.detect_anomaly():
    loss = criterion(model(x), y)
    loss.backward()
```

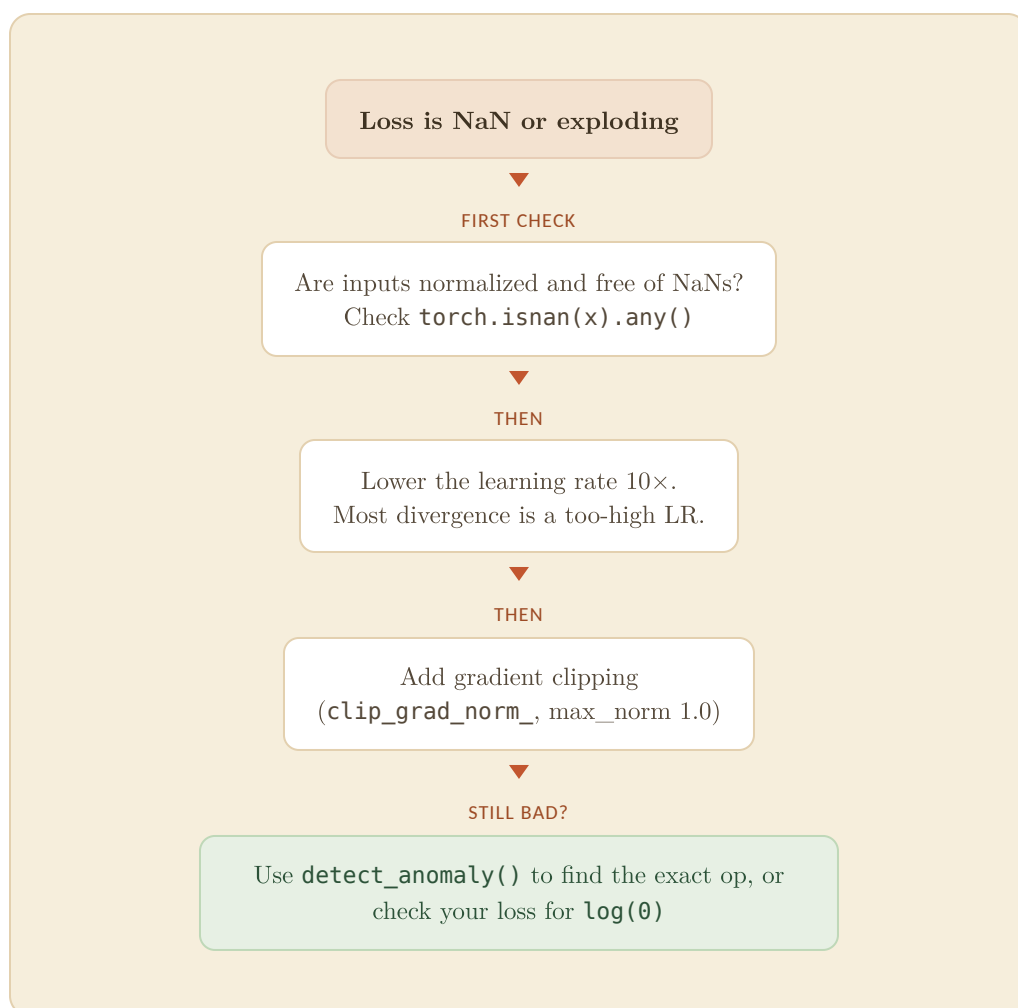
WATCH OUT

`detect_anomaly` slows training dramatically because it checks every operation. Use it to *locate* a NaN, then turn it off. Never leave it on in a real training run — it is a diagnostic tool, not a setting.

Once you have found an exploding loss, the fixes follow the cause. The most common single remedy is *gradient clipping*, which caps the size of the gradient update so one bad batch cannot blow up the weights:

```
loss.backward()
torch.nn.utils.clip_grad_norm_(
    model.parameters(), max_norm=1.0)
optimizer.step()
```

This little decision flow captures the order to try things when a loss goes bad:



A triage order for a diverging loss: rule out the cheap, common causes before the expensive ones.

Autograd and training-loop gotchas

A final family of bugs produces no error at all — the model simply refuses to learn, or learns strangely. These trace back to the mechanics of autograd and the training loop you memorized in Part V.

The most common is forgetting `zero_grad`. PyTorch *accumulates* gradients by design, so without clearing them each step, every batch's

gradient piles onto the last and your updates become nonsense. The canonical three-line order is not optional:

```
optimizer.zero_grad()    # clear old gradients
loss.backward()          # compute new ones
optimizer.step()         # apply them
```

Another silent culprit is the in-place operation. Modifying a tensor in place can overwrite a value autograd needs for the backward pass, producing either a cryptic error or a wrong gradient. When a tensor is part of the graph, prefer the out-of-place form:

```
# Risky: mutates a tensor autograd may need
x += 1
x[mask] = 0

# Safe: creates a new tensor
x = x + 1
x = x.masked_fill(mask, 0)
```

WATCH OUT

If your loss does not change at all across epochs, check three things in order: did you call `optimizer.step()`, are the parameters actually passed to the optimizer, and is anything detached from the graph with `.detach()` or a stray `no_grad()` that should not be there. A frozen loss almost always means gradients are not reaching the weights.

A repeatable debugging workflow

When something breaks, resist the urge to change code at random. Run this sequence instead, and you will find most bugs in minutes rather than hours:

THE RULE

- 1 **Read the traceback bottom-up.** Name the error, then find the deepest line in your own code.
- 2 **Shape errors:** print shapes through `forward`, or run `torchinfo.summary` to see the whole map at once.
- 3 **Device errors:** route every model and batch through one `.to(device)`.
- 4 **Dtype errors:** labels for cross-entropy must be `.long()`; features usually `float32`.
- 5 **NaN or exploding loss:** check inputs, drop the learning rate 10×, add gradient clipping, then `detect_anomaly()`.
- 6 **Model won't learn:** confirm `zero_grad` → `backward` → `step`, and that nothing is silently detached.
- 7 **Reproduce small.** Shrink to one batch and one step. A bug you can trigger in two seconds is a bug you can fix.
- 8 **Change one thing at a time.** Random edits create new bugs faster than they fix old ones.

END OF CHAPTER 34 · END OF PART VI

You can now measure what your model does, see inside it, and repair it when it breaks. These are the skills that turn code that runs into code that works — and they complete the foundations. What remains is to assemble everything you have built into real architectures, the work that begins in Volume 2.

Exercises

1. Feed a `(32, 784)` tensor into `nn.Linear(256, 10)` and trigger the shape error. Then fix it two different ways — by correcting `in_features`, and by reshaping the input — and confirm both run.
2. Build a small CNN, then add `print(x.shape)` after each layer in `forward`. Run one batch and record how the shape changes layer by layer. Where does the spatial size shrink?
3. Install `torchinfo` and run `summary` on the CNN from exercise 2 with `input_size=(1, 3, 32, 32)`. Compare its Output Shape column to your printed shapes — do they match?
4. Move a model to `"cuda"` (or simulate the error on CPU) but leave the batch on CPU. Read the device-mismatch error, then fix it. What is the minimal change?
5. Train a tiny model with an absurdly high learning rate (e.g. `lr=10`) until the loss becomes `nan`. Add the `isnan` check to catch the step. Then lower the LR and add gradient clipping — does it recover?
6. Remove `optimizer.zero_grad()` from a working training loop and watch what happens to the loss over a few epochs. Explain the pattern you see, then restore the line.
7. Wrap a training step in `torch.autograd.detect_anomaly()` and introduce a `torch.log` of a tensor containing a zero. Read where it reports the NaN originating.

INTERVIEW-STYLE PROBLEM

An interviewer says: "A colleague reports that their model's loss is stuck at exactly the same value every epoch and never moves. List the three most likely causes in order, and describe the one-line check you would run for each." Answer in five to six sentences.

If this helped

You've just read eight of thirty-four chapters. The full book carries the same idea the whole way: nothing is waved past, and every shape is worked through until it's obvious.

If any of it was useful — or if any of it wasn't — I'd honestly like to hear about it. Tell me which chapter lost you, and I'll fix it.

Free tensor cheat-sheet

<https://payhip.com/b/7ukxh>

Companion code on GitHub

https://github.com/pytorch-from-ground-up/book_code

Digital edition — Leanpub

<https://leanpub.com/pytorchfromgroundup>

Paperback — Amazon

<https://www.amazon.com/dp/B0H8WMCV33>

Volume 1 of a series working toward large language models.

Foundations first. The models come after, and they come easier.

Wesam

Author, PyTorch From Ground Up